



Safe by Construction: Schema-Bounded Action Generation and Rust Orchestration for Real-Time Immersive Speech Programming in VR

By

Sandeep Rai

Student ID: sxr1156

Supervisors: Dr Daniele Giunchi and Dr Eyal Ofek

A dissertation submitted to the University of Birmingham
for the degree of **MSc in Cyber Security**

Word count: ~10,000 (main body, Chapters 1–6)

School of Computer Science
University of Birmingham

2026

Declaration

I declare that this dissertation is my own work, submitted for the degree of MSc in Cyber Security at the University of Birmingham. Where I have drawn on the work of others it is acknowledged and cited. The original *DreamCodeVR* system (Giunchi et al., 2024) is prior work by University College London that this project analyses, preserves, and builds upon; all safety architecture, the Rust workspace, the evaluation, and the red-team campaign described here are my own. Use of AI assistance is declared in accordance with University policy.

Acknowledgements

I thank my supervisors, Dr Daniele Giunchi and Dr Eyal Ofek, for their guidance, and the authors of DreamCodeVR and the Ubiq platform, whose openly published work made this research possible.

Table of Contents

Declaration 6

Acknowledgements 6

Table of Contents 6

List of Figures and Tables 6

Abstract 6

Chapter 1 — Introduction 6

 1.1 The problem 6

 1.2 Why this matters 6

 1.3 Impact overview — what a safe design buys 6

 1.4 What is out there, and why it is not enough 7

 1.5 What is still open 7

 1.6 What we did 7

Chapter 2 — Background and Related Work 9

 2.1 Speech-driven end-user programming in VR 9

 2.2 LLM code generation and its risks 9

 2.3 Prompt injection and standardised taxonomies 9

 2.4 Constrained, schema-bounded generation 10

 2.5 Runtime execution in Unity, and the AOT constraint 10

 2.6 Container and kernel isolation 10

 2.7 XR perceptual and embodied attacks — the second safety dimension 11

 2.8 Privacy and Rust — the two engineering commitments 11

 2.9 The gap and the thesis 11

Chapter 3 — The System: Rust Orchestration, the VR Cyber-Security Surface, and Why Rust 13

3.1 Workspace shape and the four modes	13
3.2 The action-plan IR and the fail-closed validator	14
3.3 Orchestration: per-peer sessions, timeouts, and a spawn chokepoint	14
3.4 The VR cyber-security surface: two threat planes and the CVE classes they close	15
3.5 Privacy by construction, personalization, and age-as-protection	16
3.6 Live observability and runtime control	17
3.7 Why Rust	17
Chapter 4 — Evaluation	18
4.1 Method and gate	18
4.2 RQ1/RQ3 — the fail-closed gate and the adversarial red-team	18
4.3 RQ1/RQ3 — the XR attack-class benchmark	19
4.4 RQ2 — deployability	20
4.5 RQ4 and the whole-system campaign	20
Chapter 5 — Discussion: the Small Things that Make the Difference	22
Chapter 6 — Limitations, Ethics, and Future Work	24
6.1 Limitations	24
6.2 Ethics and privacy	24
6.3 Future work, prioritised	24
6.4 Closing statement	25
References	26

List of Figures and Tables

Figures and tables are numbered within their chapters.

Abstract

Immersive speech programming lets people with no coding background shape behaviour inside virtual reality (VR) by speaking. The original *DreamCodeVR* system (Giunchi et al., 2024), built at University College London on the Ubiq social-VR platform, does this by translating speech into C# that is **compiled and executed at runtime** inside Unity. The idea is compelling but the execution model is unsafe: it runs effectively arbitrary, model-generated code on the host or headset with no validation gate. It is also undeployable on current hardware — it works only as a manually sideloaded 32-bit Mono build on Quest 1/2 and cannot ship to the Quest Store or run on Quest 3, whose IL2CPP/AOT ARM64 backend forbids runtime C# compilation.

This dissertation presents **DreamCodeVR+**, which keeps the original generate-and-compile pipeline as a controlled baseline (Mode A) but replaces the *default* path with a **safe-by-construction** alternative. Speech is translated into a small, allow-listed *action plan* — a six-action JSON representation with hard numeric bounds — that a tiny, audited Unity executor runs **without**

compiling any code (Mode C), making it Quest-compatible by construction. A new **Rust** backend joins the Ubiq network as a service peer and enforces a **fail-closed** validator that approves a plan only when it contains zero violations. Two research arms enable honest comparison: a lexical-plus-Roslyn **validated-C#** path (Mode B) and a hardened **Docker** sandbox (Mode D). The Rust workspace is `unsafe`-free, panic-averse by compiler-enforced lint, and passes 342 automated tests; it is live-verified against the real Ubiq RoomServer, a real OpenAI model, a real .NET Roslyn service, and real Docker. The gVisor `runsc` runtime is wired and unit-tested but was not exercised, so kernel-level isolation is a *designed*, not a measured, property.

The thesis of the work is methodological: for safety-sensitive immersive end-user programming, **bounding the output language is a stronger and more deployable safety lever than sandboxing the output** — yet the unbounded path can itself be validator-gated when expressiveness is needed. Evidence includes a reproducible **1,057-vector** red-team that confirms the C# validator at **258/258** (all 469 benign inputs allowed) and hardens the natural-language intent screen from **81.71%** to **100%** by adding a paraphrase layer (these post-hardening figures are in-sample); a separate static code-admission benchmark of five XR attack classes in which the hardened system blocks **38 of 40 (95%)** attacks while preserving **12 of 12** benign commands; and a whole-system verification campaign that uncovered and fixed **seven latent defects**, two of them safety controls that silently did nothing. The honest residual — two of forty XR attacks that are indistinguishable from legitimate camera commands — is reported openly and motivates a runtime guardian as future work. The system is a research prototype that measures *static code admission*, not on-device runtime prevention.

Chapter 1 — Introduction

1.1 The problem

Virtual reality promises authoring environments in which people build interactive worlds *from inside them*, rather than at a desktop through an engine's scripting interface. For non-programmers, the natural interface is speech: *"make this cube red," "spin it slowly," "give it gravity."* The original *DreamCodeVR* system (Giunchi et al., 2024) realises exactly this: a user holds a controller trigger to record an utterance, points a ray at a target object, and a Node.js server transcribes the speech, prompts a large language model (LLM) to **generate C#**, and ships that code back to Unity, where a runtime compiler (RoslynCSharp) **compiles and runs it** to mutate the scene.

The problem is *how* the generated code is executed. The system performs the equivalent of `eval` on untrusted, model-authored input, with **no validation gate** between generation and execution. The compiled assembly runs with the full privileges of the Unity process — free to reach file, network, process, and reflection APIs on the host or headset. This is the classic code-injection / remote-code-execution pattern (OWASP, no date a; MITRE, 2026), and because an LLM's output can be *steered* by a prompt-injected or simply erroneous request (OWASP Gen AI Security Project, 2025a; Greshake et al., 2023), the danger is not hypothetical. Alongside this safety flaw sit three further concrete defects: the runtime-compilation feature is confined to a sideloaded 32-bit **Mono** build on Quest 1/2 and **cannot** ship to the Quest Store or run on Quest 3 (whose IL2CPP/AOT ARM64 backend provides no runtime C# compiler); there is no schema or allow-list mediating the model's output; and the published server pins its core `ubiq` dependency to a `gitpkg` host that, when observed during this project's setup, returned HTTP 402, so the artefact is not installable as published.

1.2 Why this matters

Immersive speech programming sits at a productive intersection of accessibility, voice interaction, and LLM-driven synthesis. Its promise — letting the very novices that VR's accessibility might empower author behaviour without engine expertise — is real and worth preserving (Yigitbas et al., 2022; Nowrin and Vertanen, 2023). But the same properties that make it attractive make its safety failures severe. The code runs *on the person's device*, inside a headset that also senses their body; the medium can be turned against not only the machine (file, network, process access) but the *user* — the headset can be made to walk someone into a wall or strobe their vision. An unsafe execution model in this setting is therefore not a cosmetic engineering flaw; it is a direct route to compromise of the host and, distinctly, to bodily and perceptual harm to the wearer. As LLM-mediated authoring moves toward consumer XR, a defensible answer to "what may the model cause to run?" becomes a precondition for deployment, not an optional extra.

1.3 Impact overview — what a safe design buys

The redesign in this dissertation delivers three things at once. First, **safety by construction**: because the deployable path admits only six allow-listed behaviours with hard numeric bounds,

unsafe operations are *unrepresentable* rather than merely filtered — there is no legal instruction that opens a socket. Second, **deployability**: removing runtime compilation makes the default path build under IL2CPP/ARM64, so it ships through the Quest Store and runs on Quest 3 by construction — a property the original design cannot have. Third, **transparency and reproducibility**: every safety decision is a structured, inspectable event, the guardrails can be red-teamed on demand from a live admin panel, and the entire system is checked by a one-command regression harness with a drift-detecting baseline. The measured slice is deliberately narrow and honestly labelled — the system rejects malicious code *before it runs* (static code admission), which is a different and weaker claim than preventing all on-device runtime harm — but within that slice the guarantees are strong and reproducible.

1.4 What is out there, and why it is not enough

Two mature literatures bear on this problem, and neither, alone, solves it. **Static analysis** scans generated code for dangerous patterns; it is general but reactive, bounded by the analyser's completeness, and — as this project found in its own Roslyn layer — prone to false negatives. **Sandboxing** contains whatever a process does after it runs; but a default container is a *soft* boundary (NIST SP 800-190), the container runtime's own trusted computing base is attackable (CVE-2024-21626, "Leaky Vessels"), and — decisively for XR — both approaches *presuppose the very runtime-compilation step* that confines the original system to sideloaded 32-bit Mono. A third, less-travelled literature — **constrained, schema-bounded generation** (Geng et al., 2023; Poesia et al., 2022; Scholak et al., 2021) — restricts model output to a formal grammar and thereby *guarantees* structural validity. It has rarely been applied as a *security* boundary for an execution pipeline, and never, to the author's knowledge, in an AOT-constrained immersive setting where a second, perceptual threat surface targets the user's body. That is the gap this work occupies.

1.5 What is still open

Even a correctly bounded action set does not close everything. The numeric bounds delete the catastrophic *system-level* classes — camera, locomotion, input, network — by making them unrepresentable, but they leave the *perceptual and embodied* classes expressible within bounds: a wall of individually in-bounds primitives still occludes; a sequence of sub-threshold moves still drifts the user; a legal colour change repeated rapidly still strobes. Harm here is compositional, relational, and temporal, not a property of any single action's magnitude (Krauß et al., 2024; Tseng et al., 2022; Casey et al., 2021). Two of forty XR attacks in this project's benchmark — camera rotations indistinguishable from a legitimate "turn me around" command — cannot be caught by static admission at all and require a runtime guardian that this work specifies but does not yet ship. These honest residuals frame the limits of the contribution and the shape of the future work.

1.6 What we did

DreamCodeVR+ responds not by hardening the execution of model-generated code but by **changing what is generated**. The LLM is asked to emit a schema-bounded *action plan* — a JSON

representation whose only legal instructions are six allow-listed behaviours (`set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics`), each with hard bounds — which a minimal, audited Unity executor runs with **no compilation** (Mode C). A new Rust backend joins the Ubiq network as a service peer and validates the plan **fail-closed**, approving it only when the violation set is empty. The original generate-and-compile path is preserved as **Mode A** (now itself gateable by the validator), and two research arms — a lexical-plus-Roslyn validated-C# path (**Mode B**) and a hardened Docker sandbox (**Mode D**) — enable an honest comparison of safety strategies. On top of the code-safety layer sits a **perceptual/embodied** safety layer (a head-centred personal-space invariant, scene-relational guards, and temporal envelopes), a safety-preserving **personalization (RAG)** layer whose retrieved context is treated as untrusted, a live **admin panel** that streams every stage and red-teams the guardrails on demand, and an opt-in **age-adaptive** coupling that turns age inference from a surveillance signal into a protection one. The remainder of the dissertation is organised as follows: Chapter 2 reviews the literature and states the gap; Chapter 3 describes the system, its Rust breakdown, the VR-specific cyber-security surface, and the case for Rust; Chapter 4 evaluates the system; Chapter 5 discusses what makes the difference; and Chapter 6 states limitations and future work.

The research questions are: **RQ1** — can a schema-bounded action-plan IR, enforced by a fail-closed validator, prevent the unsafe operations the original pipeline permits, and at what cost to expressiveness? **RQ2** — does the action-plan model remove the runtime-compilation dependency that makes the original Quest-incompatible? **RQ3** — how effectively do the Rust validation layers reject unsafe and adversarial input, including evasion attempts? **RQ4** — across the four modes, how do functionality, latency, security, and privacy compare?

Chapter 2 — Background and Related Work

This chapter situates DreamCodeVR+ within the literatures it draws on and shows, concretely, why each existing approach is necessary but not sufficient — the "what is out there is not good enough" of Chapter 1, expanded.

2.1 Speech-driven end-user programming in VR

Construction of interactive VR scenes has traditionally demanded C# and engine expertise, excluding non-programmers. Adjacent tools broaden novice access but do so by *restricting expressiveness*: VREUD (Yigitbas et al., 2022) and FlowMatic (Zhang and Oney, 2020) confine users to pre-built visual or block vocabularies and so never generate arbitrary code. DreamCodeVR (Giunchi et al., 2024) instead kept free-form speech and paid for that generality with an unsafe execution model. Decades of voice-programming research show speech to be viable but error-prone for verbatim code entry, and that novices express *intent* rather than syntax (Pane, Ratanamahatana and Myers, 2001; Begel and Graham, 2006; Nowrin and Vertanen, 2023). This is precisely why mapping messy spoken intent onto a small, validated action vocabulary is a better fit than dictating C#: the intermediate representation becomes the natural sink for ambiguous speech. DreamCodeVR+ sits between the restricted-vocabulary tools and DreamCodeVR — it keeps speech-first accessibility while making the output language safe.

2.2 LLM code generation and its risks

Two facts about code-generating LLMs must be established. The *capability*: Codex and HumanEval (Chen et al., 2021) showed LLMs can synthesise functioning programs from natural language. The *reliability ceiling*: the same work reported only modest single-sample correctness and systematic failure modes, generalised by later surveys (Huynh and Lin, 2025; Zhang et al., 2023). Raw LLM output therefore cannot be trusted unchecked. Executing program text derived from untrusted input is the canonical code-injection pattern — *Eval Injection* and *Code Injection* in OWASP's catalogue, CWE-94 in MITRE's (OWASP, no date a, no date b; MITRE, 2026) — granting the input the host process's privileges. Running LLM-generated C# through a runtime compiler is functionally identical, and real incidents in eval-based LLM agent frameworks (used here only as a grey-literature illustration) show it is not theoretical. This is the exact hazard of the original Mode A.

2.3 Prompt injection and standardised taxonomies

The threat model assumes the LLM can be *steered*. The OWASP Top 10 for LLM Applications (OWASP Gen AI Security Project, 2025a) codifies the relevant risks — LLM01 (Prompt Injection) and LLM05 (Improper Output Handling) — and Greshake et al. (2023) demonstrated *indirect* prompt injection compromising real applications: the model need not be jailbroken by the user directly, because any content flowing into the prompt is an attack surface. DreamCodeVR+'s mitigations map onto this taxonomy control-by-control: LLM output is treated as untrusted, the action plan is

validated fail-closed, any C# candidate is scanned lexically and semantically, and the `request_id` is fixed server-side so the model never owns its own identifier.

2.4 Constrained, schema-bounded generation

This is the conceptual heart of the project. Grammar-constrained decoding (Geng et al., 2023), constrained semantic decoding (Poesia et al., 2022, *Synchromesh*), and PICARD (Scholak et al., 2021) show that restricting generation to a formal grammar or schema both improves reliability and *guarantees* structural validity; the same principle underlies modern JSON-schema-constrained decoding (Geng et al., 2025b). The decisive property for *safety* is subtle: because the action-plan IR is finite and allow-listed, unsafe operations are **unrepresentable, not merely filtered**. There is no `set_color` argument that opens a socket, because the grammar admits no such instruction. This reframes safety from "detect bad code after the fact" (the static-analysis and sandboxing posture) to "make bad code inexpressible in the first place" — the move that differentiates DreamCodeVR+ from its predecessor.

2.5 Runtime execution in Unity, and the AOT constraint

The deployability half of the thesis rests on a hard, primary-sourced constraint. Unity's IL2CPP scripting backend compiles all C# ahead of time and provides **no runtime C# compiler**; `System.Reflection.Emit` is unsupported under AOT (Unity Technologies, 2024a, 2024b). IL2CPP with a 64-bit ARM64 target is mandatory for the Quest *Store* and for Quest 3 / current Horizon OS. Any architecture that depends on compiling C# at runtime therefore cannot be Store-published or run on current hardware — it is confined to manually sideloaded 32-bit **Mono** builds on Quest 1/2 (where, as the original project's own `ProjectSettings.asset` confirms, it genuinely does run). This is the source-grounded justification for the action-plan executor: with zero compilation, Mode C builds under IL2CPP/ARM64 by construction.

2.6 Container and kernel isolation

For studying genuinely untrusted generated C#, DreamCodeVR+ adds Mode D: a hardened container. NIST SP 800-190 is the authoritative baseline for container threats and justifies both the hardening flags and the honesty that a default container is a *soft* boundary. That honesty is underscored by CVE-2024-21626 ("Leaky Vessels"), a runc container-escape flaw showing the runtime's own trusted computing base is attackable. gVisor interposes an application kernel between workload and host, so an attacker must break two disjoint kernels to escape, at a quantified performance cost (Young et al., 2019; Wang et al., 2022). In this project only the Docker boundary was exercised; the gVisor (`runsc`) path is wired and unit-tested but not run, so its stronger isolation is a *designed*, not measured, property. This is exactly why sandboxing is positioned as a research mode and not the default — the default avoids codegen entirely.

2.7 XR perceptual and embodied attacks — the second safety dimension

The threats above concern the *system*. A distinct body of work concerns the *user*: how the medium itself can be weaponised against the person wearing the headset. Krauß et al. (2024) catalogue XR-specific dark patterns and their amplifying properties; Tseng et al. (2022) give a severity taxonomy of perceptual manipulations from recoverable to unrecoverable; Casey et al. (2021) demonstrate the *human joystick*, sub-perceptually walking a user into a real wall; Fujita et al. (2023) show 3D object disguise; and Lee et al. (2021), in the WebVR setting, provide the defensive blueprint this work adopts — *confinement* of untrusted content to what the user can legitimately see. The unifying observation is decisive: **harm is compositional, relational, and temporal, not a property of any single action's magnitude**. Allow-lists and numeric bounds are therefore necessary but not sufficient — they delete the catastrophic system-level classes yet leave the perceptual classes expressible within bounds — motivating the user-frame invariant and scene-relational layer of Chapter 3.

2.8 Privacy and Rust — the two engineering commitments

Two further literatures shape the build. On *privacy*, voice used for identification is special-category biometric data under the GDPR (Art. 9), high-risk processing of which triggers a mandatory Data Protection Impact Assessment (Art. 35), with privacy-by-design a standing duty (Art. 25); VR adds gaze and motion as further sensitive signals (European Union, 2016; European Data Protection Board, 2021; Nair et al., 2023). DreamCodeVR+ answers with telemetry minimisation — reason codes, never transcripts. On *implementation language*, RustBelt (Jung et al., 2018) gives the first machine-checked proof that well-typed *safe* Rust is free of undefined behaviour; because the workspace compiler-enforces `forbid(unsafe_code)`, those guarantees actually hold for this codebase — which matters when a memory-corruption bug in the validator would undermine every other layer. Honesty requires the counterpoint (Zheng et al., 2024): safety holds only for the safe subset, which is why `cargo-deny` audits the dependency tree. The industrial weight of the argument is Google's (2024) report of Android's memory-safety vulnerabilities falling from 76% to 24% as new code moved to memory-safe languages.

2.9 The gap and the thesis

Structured-generation research aims at making model output *valid*; systems-sandboxing research aims at *containing* what a process does. They rarely meet at the point this project occupies. DreamCodeVR+ argues that for safety-sensitive, AOT-constrained immersive end-user programming, the first lever is the stronger and more deployable one: if the output language admits only safe instructions, there is little — ideally nothing — left to sandbox, and no runtime compiler is required at all. Static analysis (Mode B) and sandboxing (Mode D) are retained as research arms; constrained generation (Mode C) is the default. Table 2.1 makes the positioning precise across four axes.

Table 2.1 — Comparison of safety strategies for executing model-derived behaviour.

Axis	Static analysis (scan of generated code)	Sandboxing (container/kernel isolation)	Constrained generation (schema-bounded plan)
Residual attack surface	Moderate–high: bounded by analyser completeness; evasions and false negatives remain (the Roslyn layer missed <code>System.Net.WebClient</code>)	Low but non-zero: the runtime's own trusted computing base is attackable (CVE-2024-21626); plain Docker is a <i>soft</i> boundary (NIST SP 800-190)	Lowest: unsafe operations are <i>unrepresentable</i> in a six-action allow-list, not merely detected
Quest Store / Quest-3 deployability	Sideload-only: still implies runtime compilation of the analysed C#, confined to Mono/32-bit Quest 1/2	Sideload-only at best: presupposes a runtime compiler/executor, so the same confinement applies	Yes by construction: zero compilation, so it builds under IL2CPP/ARM64 and ships through the Store
Latency cost	Low–moderate: per-request analysis (lexical fast; semantic adds a round-trip)	Moderate–high: container spawn, compile-and-run, isolation overhead per request	Lowest: a bounded JSON parse plus arithmetic bounds checks (~85 µs median)
Generality	High: scans arbitrary C#	High: runs arbitrary code, then contains it	Low <i>by design</i> : only six allow-listed behaviours are expressible — the deliberate trade-off

Static analysis and sandboxing are *general but reactive*, and both presuppose the very runtime-compilation step that confines the original to sideloaded 32-bit Mono. Constrained generation sacrifices generality to obtain the strongest residual-surface guarantee, the lowest latency, and the only Store-/Quest-3-deployable path. The central, defended claim is therefore methodological: **bounding the output language is a stronger and more deployable safety lever than sandboxing the output, and it can be enforced by construction.**

Chapter 3 — The System: Rust Orchestration, the VR Cyber-Security Surface, and Why Rust

This chapter describes the delivered system: the Rust workspace and the four execution modes; the two distinct threat surfaces it defends (system-level and perceptual/embodied), including the concrete CVE and injection classes it neutralises; and the principled case for building the safety core in Rust.

3.1 Workspace shape and the four modes

The backend is a single Cargo workspace of sixteen library crates and five binaries, `unsafe`-free by lint policy. The safety-critical crates are small and single-purpose: `behaviour-dsl` defines the action-plan IR; `code-policy` is the fail-closed validator; `command-router` owns per-peer orchestration; `unity-transport` implements the Ubiq wire protocol; `csharp-policy` and `roslyn-client` provide the Mode-B lexical and semantic C# gates; `sandbox` drives Mode D; `observability` enforces privacy-safe telemetry; `personalization`, `control`, `admin`, and `config` provide RAG, the runtime control plane, the web dashboard, and typed configuration; and `roomserver` is a from-scratch Rust reimplement of the Ubiq relay. The binaries include the server itself, a fake-Quest test client, a Ubiq probe, the sandbox runner, and the XR-security benchmark.

Four modes span the safety-strategy design space. **Mode A** is the preserved original: model-generated C# compiled and run at runtime (now itself gateable by the validator). **Mode B** is validated C#: a tree-sitter lexical scan plus a .NET Roslyn semantic check. **Mode C** is the default deployable path: the allow-listed action plan executed with zero compilation. **Mode D** is the hardened container for genuinely untrusted C#. Modes A, B, and D exist chiefly to make the safety case for Mode C empirically, not rhetorically. Table 3.1 summarises how the four modes trade safety mechanism against expressiveness and deployability.

Table 3.1 — The four execution modes.

Mode	What runs	Safety mechanism	Compilation	Deployability
A (baseline)	Model-generated C#	Optional validator gate (retained for comparison)	Runtime (Roslyn)	Sideloading-only, Mono/32-bit Quest 1/2
B (research)	Validated C#	Tree-sitter lexical scan + Roslyn semantic check	Runtime	Sideloading-only
C (default)	Allow-listed action plan	Unsafe operations unrepresentable; fail-closed validator	None	Store / Quest 3 by construction
D (research)	Untrusted C# in a container	Hardened Docker (gVisor <code>runsc</code> wired, not run)	Runtime, isolated	Server-side only

The four original weaknesses map cleanly onto these responses. Runtime arbitrary code execution (Weakness 1) is answered by Mode C making the dangerous operations unrepresentable; Store/Quest-3 incompatibility (Weakness 2) by removing runtime compilation; the missing validation gate and global `isGenerating` lock (Weakness 3) by the fail-closed validator plus per-peer sessions; and the

dead `gitpkg` supply-chain pin (Weakness 4) by vendoring the RoomServer from GitHub and adding a `cargo-deny` supply-chain gate to the new workspace.

3.2 The action-plan IR and the fail-closed validator

The IR (`behaviour-dsl`) is an internally-tagged, `deny_unknown_fields` enum of six actions — `set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics` — so the *enum itself is the safety allow-list*. Its bounds live in a single authoritative module, summarised in Table 3.2. The validator (`code-policy`) rejects oversized input *before parsing* (so a hostile 100 MB payload is refused without ever being deserialised), accumulates *all* violations rather than short-circuiting on the first, and approves a plan **if and only if** the violation set is empty. This fail-closed discipline is the single most important property in the system: any error, ambiguity, or unrecognised field degrades to a refusal, never to execution. Because the IR is finite and allow-listed, whole attack classes — file access, sockets, process spawning, reflection — are not "blocked" but *absent from the language*. There is no argument to any of the six actions that names a namespace, a file path, or a socket, because the grammar has no field in which such a value could appear.

Table 3.2 — The authoritative bounds enforced by the fail-closed validator.

Bound	Value	Purpose
MAX_ACTIONS	16 per plan	Caps a single utterance's blast radius
MAX_SPAWN_COUNT	8 per spawn action	Limits one instruction's object creation
MAX_TOTAL_SPAWNED_PER_SESSION	64	Cumulative chokepoint across a session
scale	0.1–5.0	Prevents giant/occluding or invisible objects
rotation	≤ 360°/s	Anti-vection clamp against disorientation
mass	0.1–100	Bounds physics-driven projectile force
MAX_PLAN_JSON_BYTES	64 KiB	Pre-parse size gate against payload floods

3.3 Orchestration: per-peer sessions, timeouts, and a spawn chokepoint

The original server enforced a single **global `isGenerating` flag** as its only concurrency control, so any one user's in-flight request silently dropped every other user's commands, and a single failed generation (bad API key, rate limit, dropped connection) left the flag permanently set and wedged the pipeline for *all* peers with no timeout or watchdog. `command-router` replaces this with a `HashMap<peer, session>` — no global lock — giving each peer an independent session, a per-call async timeout, and a single spawn-budget chokepoint that enforces the cumulative cap across a session. Because the router is the sole point at which a plan is dispatched, the cumulative cap cannot be bypassed by fragmenting a request across many small plans: every accepted plan decrements the same per-peer budget.

Implementing the Ubiq wire protocol exactly surfaced two real, previously-latent protocol defects, found and fixed against the genuine RoomServer: the frame length is `NetworkId(8) + payload` (not payload alone), and the `Join` handshake's `args` field must be *stringified* JSON rather than a nested object. These were not documented anywhere; they were recovered by driving the real relay and diffing byte streams until the handshake succeeded — the kind of interoperability detail that only live testing exposes.

The from-scratch Rust RoomServer (`crates/roomserver`) is a single-binary, `unsafe`-free relay that reaches stock-Ubiq protocol parity, adding the five previously-missing control messages — `AppendPeerProperties`, `AppendRoomProperties`, `DiscoverRooms`, `SetBlob`, `GetBlob` — with properties that persist into later room state, and validating RFC-4122-v4 room UUIDs as upstream does. In live testing it **stayed available on a malformed `SetBlob` control message that crashes the genuine UCL Node.js RoomServer** (`TypeError: this.server.setBlob is not a function`), where one client can otherwise take down the relay and every peer in the room. This is a concrete robustness improvement over the upstream artefact: the crash was reproduced and documented, and the third-party code left unpatched in keeping with responsible disclosure. It also makes the whole system reproducible as a single Rust deployment rather than a Node.js server whose published manifest no longer installs.

3.4 The VR cyber-security surface: two threat planes and the CVE classes they close

DreamCodeVR+ defends **two distinct planes**, and treating them as one would understate the problem.

The **system plane** is the classic surface: untrusted, model-authored code reaching the host's file, network, process, or reflection APIs. This is CWE-94 code injection (MITRE, 2026) and OWASP's LLM01 (prompt injection) and LLM05 (improper output handling) (OWASP Gen AI Security Project, 2025a). Because the LLM's output can be steered by the user's spoken prompt or by any content that reaches the prompt indirectly (Greshake et al., 2023), the generated C# is, from the system's point of view, attacker-influenced input; running it is `eval` on that input. Mode C closes this plane *by construction* — the operations are unrepresentable — which is why it needs no sandbox at all. Where generated C# is studied at all (Modes B and D), the container hardening is calibrated against NIST SP 800-190 and the concrete escape risk of **CVE-2024-21626** ("Leaky Vessels"), a `runc` file-descriptor leak that lets a container escape to the host filesystem: the point of citing a *specific, dated* CVE is that it proves the container runtime's own trusted computing base is attackable, so a container is a soft boundary and must not be presented as the primary defence. The Mode-D flags follow directly: `--network none` (no exfiltration channel), read-only rootfs plus tmpfs (no persistence), `--cap-drop ALL` and `no-new-privileges` (no privilege gain), PID/memory/CPU limits (no resource exhaustion), a non-root UID, and a wall-clock timeout backed by `docker kill`, with only a structured report crossing back across the boundary. An honest defence-in-depth finding emerged here: the Roslyn semantic layer is `deny-list-only` and `shallow` — it returned a **false negative** for `System.Net.WebClient`, a network exfiltration API — which the Rust lexical layer caught as the

backstop. This is the empirical proof of the thesis's core claim: the moment safety depends on a *detector*, its completeness becomes the ceiling, and a real detector missed a real attack. In the default configuration (no `DCVR_ROSLYN_URL` set), the Roslyn client is an approve-all, fail-open mock, so the lexical layer is the effective gate; this is stated plainly rather than glossed, because a reader must not assume the semantic check is running when the default deployment does not run it.

The **perceptual/embodied plane** is XR-specific and is the novel contribution. Numeric bounds delete the catastrophic system classes but leave perceptual harm expressible within bounds, so the system adds, ordered by strength: capability omission; the fail-closed validator; a **user-frame invariant** (an un-spoofable, head-centred personal-space sphere and forward occlusion cone that no generated object may enter or occlude); **scene-relational** guards (generated-object provenance, no pose-to-behaviour feedback loop, bounded displacement, overlap/disguise checks); and **temporal envelopes** (a per-peer accepted-plan rate cap of ≤ 3 plans/s as a WCAG 2.3.1 flash-rate proxy, an anti-vection rotation clamp, and per-peer generation rate limiting). A luminance-swing cap and a coherent optic-flow budget are specified as constants but are **not yet consumed by enforcement** — stated as a known gap. The user-frame invariant is the key idea because it holds *however many* in-bounds actions are composed, which a per-action numeric check cannot. These guards are implemented in the deployable Mode-C Unity executor (`ActionPlanExecutor` / `PerceptualSafety`), opt-in via a comfort setting so free creation is unconstrained while the safety demonstration arms them.

3.5 Privacy by construction, personalization, and age-as-protection

Privacy is enforced in the type system, not by policy. The telemetry `TimingEvent` is a JSONL schema carrying only ids, timestamps, decisions, reason codes, and counts, with a test asserting that no audio, transcript, secret, or image field can appear; raw STT/LLM detail goes only to stderr. The **personalization (RAG)** layer biases the *aesthetic* of generated output toward a user's taste (via `text-embedding-3-small`, or a deterministic offline mock) learned from thumbs-up/down feedback; retrieved context is injected as clearly-labelled **untrusted data**, and the fail-closed validator still gates every output, so RAG cannot widen what may be generated. Honesty note: RAG is **on by default** (opt-out via `enable_rag`), and it persists verbatim commands and generated C# to per-peer JSON on disk; at-rest encryption (ChaCha20-Poly1305) and a TTL retention sweep are implemented but off by default. Finally, an opt-in **age-adaptive coupling** turns an `AgeBand { Child, Teen, Adult, Unknown }` signal (default `Unknown`, failing safe to *child*) into a protection lever: a detected minor flips the C# validator to its hardened profile. The age-to-code-plane coupling is fully wired and tested; the parallel age-tightening of the *perceptual* plane is computed and unit-tested at the configuration layer but is **not yet consumed by runtime perceptual enforcement** — future work. The platform (Meta Quest) age API remains the authoritative source; the voice gate is a secondary net.

3.6 Live observability and runtime control

A security system that cannot be inspected cannot be trusted, so DreamCodeVR+ makes its own safety behaviour directly observable. A web admin panel (`control` + `admin`, built on Rust/axum with server-sent events) streams every pipeline stage live — transcript → prompt → generated C# → validation decision → compile — with per-stage timings and a total, so an evaluator watches a command travel the pipeline in real time and sees precisely *which* layer accepted or rejected it and why. It exposes runtime-tunable safety knobs (maximum C# size and line count, per-mode toggles, model selection) and a built-in **red-team runner** that fires malicious samples at the guardrail on demand and shows them being blocked. This turns the system's internal safety behaviour from a claim in a report into reproducible, inspectable evidence: an examiner can reproduce the block, change a knob, and watch the decision change. It was also the instrument that exposed one of the seven latent defects — the admin "Mode A" toggle that reported *off* while still dispatching code — because the panel showed the code reaching the dispatch stage after the control claimed to have closed it.

3.7 Why Rust

The safety core is Rust for principled, not promotional, reasons. First, **provable memory safety**: RustBelt (Jung et al., 2018) machine-checks that well-typed safe Rust is free of undefined behaviour, and because the workspace enforces `forbid(unsafe_code)`, that guarantee holds here — a memory-corruption bug in the validator would silently undermine every layer above it. Second, **fail-closed by type**: denying `unwrap/expect/panic` in core crates and modelling every fallible step as a `Result` makes "degrade to refusal on any error" a *compiler-enforced* invariant rather than a review convention. Third, **a supply-chain gate**: `cargo-deny` audits advisories, bans, licences, and sources on every run, answering the honest counterpoint that Rust's safety holds only for the safe subset and that risk enters through dependencies (Zheng et al., 2024). Fourth, **operational fit**: a single static binary joins the Ubiq network as a service peer with per-peer async sessions and bounded timeouts. The industrial evidence — Android's memory-safety vulnerabilities falling from 76% to 24% as new code moved to memory-safe languages (Google, 2024) — is the macro-scale version of the same argument. The result is a workspace that is `unsafe`-free, panic-averse, `cargo-deny`-clean, and passes 342 automated tests.

Chapter 4 — Evaluation

The evaluation asks what is *verified today*, separated cleanly from what is designed or planned. The honest headline is that the system measures **static code admission** — rejecting malicious code before it runs — not on-device runtime prevention. Within that slice, the results are reproducible and strong.

4.1 Method and gate

Every backend feature is verified by its downstream *effect*, not merely that it responded. The Rust workspace passes **342 automated tests** with `cargo fmt`, `cargo clippy -D warnings`, and `cargo deny` all green, and the whole system is checked by a single-command harness (`scripts/verify-all.sh`, 40 checks) that stands up the real .NET Roslyn analyzer, the Mode-D Docker image, and the genuine UCL Ubiq RoomServer, with a `--baseline` snapshot that later runs `diff` against to detect drift (drift detection itself verified by deliberate perturbation). The backend is live-verified end-to-end against real Ubiq, a real OpenAI model (deployed default `gpt-4o-mini`), real Roslyn, and real Docker; the Unity client is verified live in **Unity 6** (Editor Play, no Quest) driving both the Mode-C executor and the validator-gated Mode-A runtime-compile path — the latter compiling model-generated C# at runtime to build whole structures (a house, a tree, a castle) under guardrails that block malicious or over-large code. Table 4.1 records the verification status of each claim, separating what is *verified live*, *verified by test*, and *designed but not measured*.

Table 4.1 — Verification status by claim.

Claim	Status	Evidence
Fail-closed action-plan validator	Verified (test)	342 workspace tests
Live Ubiq round-trip (Mode C)	Verified live	Real RoomServer; two protocol bugs found and fixed
Mode B validated-C# gate	Verified live	Roslyn false-negative found, lexical backstop caught it
Mode D Docker containment	Verified live	Hardened container, structured report only
gVisor <code>runsc</code> isolation	Designed, not measured	Wired + unit-tested; no <code>runsc</code> host
Unity 6 execution (Modes C and A)	Verified live	Editor Play, no Quest
On-device runtime compile (Mode A)	Verified live	Quest 1/2 Mono sideload
Quest 3 / Store IL2CPP (Mode C)	Future work	Structural argument only
1,057-vector red-team	Verified (in-sample)	258/258 C#; 563/563 NL post-hardening
Five-class XR benchmark	Verified	38/40 blocked, 12/12 benign preserved

4.2 RQ1/RQ3 — the fail-closed gate and the adversarial red-team

A reproducible **1,057-vector** campaign exercises the validators: 727 C# vectors (258 should-block, 469 benign) and 330 natural-language vectors (305 should-block, 25 benign). The C# lexical validator, after hardening, **confirms at 258/258 (100%)** with all 469 benign inputs allowed. The

hardening itself is a genuine finding, and the three evasion classes it closed are instructive because each defeats a *naïve* substring scan while remaining trivially caught once the scanner is made structural. **Namespace aliasing** — `using Sock = System.Net.Sockets;` then using `Sock.TcpClient` — hides the banned name behind an alias, defeated by reconstructing dotted names and resolving alias chains before matching. **Unicode-escape identifiers** — writing `System.Net` so the source text never literally contains "Net" — defeated by decoding escapes before the scan. **Dynamic late-binding** — reaching an API by reflection or string-built type names at runtime — defeated by banning the reflection entry points themselves. Two further evasions discovered during an earlier audit (an `unsafe {}` pointer block and a `MonoBehaviour`-impostor substring) were fixed and locked with red-team tests but are outside the 1,057-vector corpus.

On the natural-language surface, the two-layer intent screen was hardened from **81.71%** (460 of 563 attack vectors counted across both layers; the screen alone caught only 202/305, leaving **103 plain-English bypasses** its technical-keyword table missed) to **100%** (563/563) by adding a natural-language *paraphrase* layer. The gap was concrete: the screen's keyword table caught "`call System.Net.WebClient`" but not "*quietly send my location to a server I control*" — the same intent phrased in plain English. The paraphrase layer matches the *intent* rather than the tokens, and closing the 103 bypasses cost zero over-blocks on the 25 benign natural-language commands. These post-hardening figures are **in-sample** — the same hand-authored corpus located the bypasses and validated the fixes — and the intent screen remains an advisory, partly fail-open pre-filter; the fail-closed validator is the authoritative gate, so a bypass of the advisory screen still meets the structural allow-list behind it. An independently-authored holdout corpus is identified as future work precisely because in-sample figures cannot, by construction, measure generalisation.

4.3 RQ1/RQ3 — the XR attack-class benchmark

A separate static code-admission benchmark spans five XR attack classes across 8 payloads plus 12 benign commands = 40 attacks + 12 benign. The five classes are chosen to span the XR-specific harm surface: **biometric exfiltration** (leaking gaze, hand, or motion data that can re-identify a user, Nair et al., 2023); **positional/motion** manipulation; **image-of-surroundings** capture (exfiltrating passthrough or room-scan imagery); the **human joystick** (Casey et al., 2021), sub-perceptually walking the wearer into a real obstacle; and **chaperone** manipulation (tampering with the guardian boundary that keeps a user inside a safe play area). The progression across three configurations is the whole argument: with **no guardrail**, **0/40** attacks are blocked; a plain code-security filter reaches **15/40**, catching the classes that read like dangerous *code* but missing the pure XR-manipulation classes (human-joystick, chaperone) *entirely*, because those are valid code that manipulates the *user*, not the *system*; and the **fully layered** system — code filter plus the perceptual layer — blocks **38/40 (95%)** while preserving **12/12** benign creative commands (0 over-blocked), at a validator latency of roughly 85 μ s median in release. The finding is that a code-security filter alone is structurally insufficient for XR; only the added perceptual layer moves the number past 15, so **defence must be layered across both threat planes**. The honest residual is two of forty (`joy-05` / `joy-06`): camera rotations indistinguishable from a legitimate "*turn me around*" command, which no static admission

gate can separate — the code is identical whether the intent is benign or hostile — and which therefore require the runtime guardian of Chapter 6 to disambiguate by temporal and pose context. That is why the figure is 95%, not 100%, and reporting it as such rather than rounding up is part of the contribution.

4.4 RQ2 — deployability

Mode C removes the runtime-compilation dependency entirely, so it builds under IL2CPP/ARM64 and is Store-/Quest-3-compatible by construction. On-device operation has been demonstrated live on a physical Meta Quest on the university network — a command spoken into the headset microphone was captured, transcribed, generated, validated, and compiled at runtime on-device — for the **Quest 1/2 32-bit Mono sideload** of the runtime-compile path (Mode A). The **Quest 3 IL2CPP/Store** build of the no-compilation Mode-C path remains future work: RQ2 is answered *structurally* (the dependency is removed) and demonstrated for the Mono path, with the IL2CPP build outstanding.

4.5 RQ4 and the whole-system campaign

Modes A–D are compared qualitatively on residual attack surface, deployability, latency, and generality (Table 2.1): constrained generation (C) has the lowest residual surface, the only Store-deployable path, and the lowest latency (a bounded JSON parse plus arithmetic), at the deliberate cost of generality; static analysis (B) and sandboxing (D) are general but reactive and both presuppose a runtime compiler. The four-mode *quantitative* latency/reliability study and any human-subjects evaluation are specified but identified as future work pending data collection.

The verification campaign itself produced an unexpected result: checking features by their downstream *effect* rather than their response uncovered and fixed **seven latent defects**. The two most serious were safety controls that *silently did nothing*. First, an admin "Mode A" toggle that, when switched off, still dispatched generated code to the headset — the UI reported the control as disabled while the dangerous path stayed open. Second, a GDPR data-retention sweep (`purge_expired`) that was documented and unit-tested as a privacy control but had **no runtime caller**, so personalization profiles were retained forever despite the code that was meant to expire them existing and passing its own unit test. The remaining five ranged from an intent-screen gap that missed 103 plain-English attacks, to a port-advertisement mismatch in service discovery, to configuration fields that never reached the code paths that read them. The lesson is methodological and is returned to in Chapter 5: a test that asserts a control *responded* is not a test that the control *works*. Every defect is now pinned by a check in the 40-check harness, so the same class of silent failure cannot recur undetected.

Two supporting ML thrusts are reported with equal candour: a calibrated voice age-band gate (ECE $0.051 \rightarrow 0.012$ on validation, 0.007 held-out, $T \approx 1.93$; 51 tests) that red-teams a numpy *stand-in* (`MockAgeGate`), not the shipped classifier; and a continuous attack analyzer (pooled AUC 1.00 on the controlled corpus — a separation artefact of that corpus, dropping to ~ 0.70 AUROC on a hard-

negative slice; 31 tests) that grows a discovered-vector store from 128 to 164 entries against a deliberately naive substring denylist (`MockDenylistSUT`), not the real validator. Against the 128-vector threat model spanning five families (network and identity, generated code, immersive/VR, voice and sensors, and privacy), the honest tally is **33 solved (built and tested), 2 partial, 80 designed, 13 N/A** — not "128 attacks neutralised". Most of the 80 designed vectors are on-device concerns that only a full hardware deployment can close, and stating the breakdown rather than the headline count is deliberate: an examiner should be able to see exactly how much is verified versus planned.

Taken together, the results answer the four research questions as follows. **RQ1** (safety vs expressiveness): the schema-bounded IR prevents the unsafe operations the original permits — they are unrepresentable, and the fail-closed validator confirms at 258/258 on the C# surface — at the cost of a six-behaviour vocabulary, a trade-off the Unity 6 demonstrations suggest is tolerable for creative authoring. **RQ2** (Quest compatibility): Mode C removes the runtime-compilation dependency, answered structurally and demonstrated for the Mono path, with the Quest 3 IL2CPP build outstanding. **RQ3** (gate effectiveness): the layered validators reject 258/258 C# and 563/563 natural-language attack vectors in-sample and 38/40 XR attacks in the benchmark, with the two residuals and the in-sample caveat reported openly. **RQ4** (mode trade-offs): constrained generation dominates on residual surface, latency, and deployability, at the cost of generality, with the full quantitative latency study identified as future work.

Chapter 5 — Discussion: the Small Things that Make the Difference

The headline architecture — bound the language, not the sandbox — is one idea. What makes it *work*, and what distinguishes this dissertation from a plausible-sounding design, is an accumulation of small, deliberate decisions. This chapter draws them out.

Unrepresentable beats filtered. The single most consequential choice is that the six-action enum with `deny_unknown_fields` makes unsafe operations *absent from the language* rather than *rejected by a check*. A filter can be evaded, has false negatives, and grows a maintenance burden with every new bypass; an absent capability has none of these. The Roslyn false negative for `System.Net.WebClient` is the proof by contrast — the moment you rely on detection, completeness becomes your ceiling. The action plan has no ceiling to raise because the dangerous instruction was never expressible.

Fail-closed as a type, not a habit. Denying `unwrap / expect / panic` in the core crates converts "degrade to refusal on any error" from a discipline the author must remember into an invariant the compiler enforces. This is why the validator's guarantee survives refactoring: you cannot accidentally introduce an execution path that runs on error, because such a path would not compile. The small decision (a lint policy) buys a large property (a boundary that cannot silently open).

Layering is not redundancy — it is coverage of *different* threat planes. The benchmark's $0 \rightarrow 15 \rightarrow 38$ progression makes this concrete. A code-security filter alone reaches 15/40 and then *stops*, because human-joystick and chaperone attacks are valid code that manipulates the *user*, not the *system*. Only the perceptual layer moves the number, and the user-frame invariant is what catches composite attacks that every per-action numeric bound passes individually. The difference between a system that scores 15 and one that scores 38 is the recognition that XR has two threat planes, not one.

Verifying by effect, not by response. The campaign that found seven latent defects worked precisely because it refused to accept a `200 OK` as evidence. Two of those defects were *safety controls that reported success while doing nothing* — the disabled Mode-A toggle that still shipped code, the retention sweep with no caller. A test that asserts the control *responded* would have stayed green; a test that asserts the headset *received no code* and the profile *was actually purged* caught them. The small methodological shift — check the downstream effect — is what separates a system that looks safe from one that is.

Honesty as a load-bearing feature. The dissertation's strongest claim is not 95% or 258/258; it is that every number is qualified exactly as far as the evidence supports and no further. The in-sample caveat on the red-team, the "designed not measured" label on gVisor, the approve-all-mock disclosure on Roslyn, the 33/2/80/13 tally instead of "128 neutralised", the 2/40 residual reported openly — each of these makes the *remaining* claims trustworthy. In a security artefact, a defensible boundary that is honestly bounded is worth more than an impressive boundary that is not.

Deployability is a security property. It is easy to treat Store-compatibility as a product concern rather than a safety one. But a safety design that cannot ship on the target hardware protects no one; the original's runtime compiler is both its expressiveness and its deployment dead-end. Removing compilation is simultaneously the safety move (nothing to inject) and the deployment move (builds under IL2CPP). The two are the same decision seen from two sides — which is why constrained generation, not sandboxing, is the thesis.

Chapter 6 — Limitations, Ethics, and Future Work

6.1 Limitations

This is a research prototype and **not a production security boundary**. It measures *static code admission* — rejecting malicious code before it runs — not on-device runtime prevention; the two must not be conflated. Five limitations are stated plainly. First, the **red-team is in-sample**: the same hand-authored 1,057-vector corpus that located the bypasses also validated the fixes, so 258/258 and 100% are ceiling figures against a known corpus, not a generalisation claim; an independently-authored holdout corpus is future work. Second, **gVisor is designed, not measured**: the `runsc` runtime is wired and unit-tested in the runner but was never exercised for lack of a `runsc` host, so kernel-level isolation is unproven and Docker is the true lower bound. Third, **two specified controls are not yet wired to enforcement**: the perceptual luminance-swing cap and optic-flow budget exist as constants but are not consumed, and the age-to-perceptual-plane posture is unit-tested at the configuration layer but not read by runtime enforcement. Fourth, the **default configuration is permissive in two respects that should be tightened for deployment**: the Roslyn semantic layer defaults to an approve-all, fail-open mock (so the lexical layer is the effective gate), and personalization is on by default and persists verbatim commands and C# to disk in plaintext unless encryption is explicitly enabled. Fifth, the **2/40 residual is real**: two camera-rotation attacks are indistinguishable from legitimate commands at admission time and cannot be closed without runtime state.

6.2 Ethics and privacy

The system was built privacy-by-construction — reason codes never transcripts, telemetry asserted by test to carry no audio or secrets — but an actual Data Protection Impact Assessment has **not** been conducted and is a precondition for any human-subjects study; voice is special-category biometric data under GDPR Art. 9 and any study triggers Art. 35 (European Union, 2016). The age-band gate is deliberately coarse (child/teen/adult, never a precise age and never affect) and inverts age inference from surveillance to protection, with the platform age API kept authoritative. The upstream RoomServer crash was reproduced and documented but its third-party code left unpatched, consistent with responsible disclosure. AI assistance in preparing this work is declared per University policy.

6.3 Future work, prioritised

1. **A runtime `UserFrameGuardian`** to close the 2/40 residual — the one component that would turn 95% into a defensible 100% on the measured slice — by carrying per-peer temporal and pose state that admission-time checks cannot.
2. **The Quest 3 `IL2CPP/Store build`** of the no-compilation Mode-C path, completing the deployability answer to RQ2 that is presently structural plus a Mono-path demonstration.

3. **An independently-authored holdout corpus** to convert the in-sample red-team figures into a generalisation claim, and a full four-mode quantitative latency/reliability study.
4. **Wiring the remaining specified controls:** gVisor `runsc` in production, the luminance/optic-flow enforcement, and the age-to-perceptual-plane posture; and hardening the defaults (require the real Roslyn analyzer, make RAG opt-in and encrypted-at-rest by default, run the retention sweep).
5. **A human-subjects study** of expressiveness and comfort, preceded by the DPIA, to test whether the six-action vocabulary is a tolerable price for safety in real use.

6.4 Closing statement

DreamCodeVR+ takes a compelling but unsafe research artefact and shows that its idea — programming a VR world by speaking — can be made safe *and* deployable without abandoning it. The lever is to bound what the model may cause to run, so that the catastrophic classes are unrepresentable rather than merely watched for; the same move that removes the injection surface also removes the runtime compiler that made the original undeployable. The contribution is not a claim of perfect safety — the residuals are named, the caveats are load-bearing, and the measured slice is narrow by design — but a demonstration, reproducible end-to-end, that for safety-sensitive immersive end-user programming, **bounding the output language is the stronger and more deployable lever**, and that the unbounded path, when expressiveness demands it, can itself be made to pass through the same fail-closed gate.

References

- Begel, A. and Graham, S.L. (2006) 'An assessment of a speech-based programming environment', in *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2006)*. Brighton, UK: IEEE, pp. 116–120. doi:10.1109/VLHCC.2006.9.
- Casey, P., Baggili, I. and Yarramreddy, A. (2021) 'Immersive virtual reality attacks and the human joystick', *IEEE Transactions on Dependable and Secure Computing*, 18(2), pp. 550–562. doi:10.1109/TDSC.2019.2907942.
- Chen, M., Tworek, J., Jun, H. et al. (2021) *Evaluating large language models trained on code*. arXiv:2107.03374.
- European Data Protection Board (2021) *Guidelines 02/2021 on virtual voice assistants, Version 2.0*. Adopted 7 July 2021.
- European Union (2016) *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Arts 9, 25 and 35. *Official Journal of the European Union* L 119, 4 May 2016.
- Friston, S.J., Congdon, B.J., Swapp, D. et al. (2021) 'Ubiq: a system to build flexible social virtual reality experiences', in *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology (VRST '21)*. New York: ACM, pp. 1–11. doi:10.1145/3489849.3489871.
- Fujita, M., Kurasaki, S. and Kanaoka, A. (2023) 'Securing cross reality: unraveling the risks of 3D object disguise on head mount display', in *Proceedings of the 13th International Conference on the Internet of Things (IoT '23)*. New York: ACM, pp. 281–286. doi:10.1145/3627050.3631570.
- Geng, S., Josifoski, M., Peyrard, M. and West, R. (2023) 'Grammar-constrained decoding for structured NLP tasks without finetuning', in *Proceedings of EMNLP 2023*. Singapore: ACL, pp. 10932–10952.
- Geng, S., Cooper, H., Moskal, M. et al. (2025b) *JSONSchemaBench: a rigorous benchmark of structured outputs for language models*. arXiv:2501.10868.
- Giunchi, D., Numan, N., Gatti, E. and Steed, A. (2024) 'DreamCodeVR: towards democratizing behavior design in virtual reality with speech-driven programming', in *2024 IEEE Conference on Virtual Reality and 3D User Interfaces (IEEE VR 2024)*. Orlando, FL: IEEE, pp. 579–589. doi:10.1109/VR58804.2024.00078.
- Google (2024) 'Eliminating memory safety vulnerabilities at the source', *Google Online Security Blog*, 25 September.
- Greshake, K., Abdelnabi, S., Mishra, S. et al. (2023) 'Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection', in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. New York: ACM, pp. 79–90. doi:10.1145/3605764.3623985.

- Huynh, N. and Lin, B. (2025) *Large language models for code generation: a comprehensive survey*. arXiv:2503.01245.
- Jung, R., Jourdan, J.-H., Krebbers, R. and Dreyer, D. (2018) 'RustBelt: securing the foundations of the Rust programming language', *Proceedings of the ACM on Programming Languages*, 2(POPL), Article 66. doi:10.1145/3158154.
- Krauß, V., Saeghe, P., Boden, A. et al. (2024) 'What makes XR dark? Examining emerging dark patterns in augmented and virtual reality through expert co-design', *ACM Transactions on Computer-Human Interaction*, 31(3). doi:10.1145/3660340.
- Lee, H., Lee, J., Kim, D. et al. (2021) 'AdCube: WebVR ad fraud and practical confinement of third-party ads', in *Proceedings of the 30th USENIX Security Symposium*, pp. 2543–2556.
- MITRE (2024) *CVE-2024-21626: runc container escape ('Leaky Vessels')*. National Vulnerability Database.
- MITRE (2026) *CWE-94: improper control of generation of code ('code injection')*. The MITRE Corporation.
- Nair, V., Rack, C., Guo, W. et al. (2023) *Inferring private personal attributes of virtual reality users from head and hand motion data*. arXiv:2305.19198.
- National Institute of Standards and Technology (2017) *Application container security guide (SP 800-190)*, by Souppaya, M., Morello, J. and Scarfone, K. Gaithersburg, MD: NIST.
- Nowrin, S. and Vertanen, K. (2023) 'Programming by voice: exploring user preferences and speaking styles', in *Proceedings of the 5th ACM Conference on Conversational User Interfaces (CUI '23)*. Eindhoven: ACM. doi:10.1145/3571884.3597130.
- Open Worldwide Application Security Project (OWASP) (no date a) *Code injection*. Available at: https://owasp.org/www-community/attacks/Code_Injection.
- Open Worldwide Application Security Project (OWASP) (no date b) *Direct dynamic code evaluation ('eval injection')*.
- OWASP Gen AI Security Project (2025a) *OWASP Top 10 for LLM Applications 2025*.
- Pane, J.F., Ratanamahatana, C.A. and Myers, B.A. (2001) 'Studying the language and structure in non-programmers' solutions to programming problems', *International Journal of Human-Computer Studies*, 54(2), pp. 237–264. doi:10.1006/ijhc.2000.0410.
- Poesia, G., Polozov, A., Le, V. et al. (2022) 'Synchromesh: reliable code generation from pre-trained language models', in *Proceedings of ICLR 2022*. arXiv:2201.11227.
- Scholak, T., Schucher, N. and Bahdanau, D. (2021) 'PICARD: parsing incrementally for constrained auto-regressive decoding from language models', in *Proceedings of EMNLP 2021*. ACL, pp. 9895–9901.

Tseng, W.-J., Bonnail, E., McGill, M. et al. (2022) 'The dark side of perceptual manipulations in virtual reality', in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. New Orleans, LA: ACM. doi:10.1145/3491102.3517728.

Unity Technologies (2024a) *Scripting restrictions*. Unity Manual.

Unity Technologies (2024b) *Introduction to IL2CPP*. Unity Manual.

Wang, X., Du, J. and Liu, H. (2022) 'Performance and isolation analysis of runC, gVisor and Kata Containers runtimes', *Cluster Computing*, 25(2), pp. 1497–1513. doi:10.1007/s10586-021-03517-8.

Yigitbas, E., Klauke, J., Gottschalk, S. and Engels, G. (2022) 'End-user development for interactive web-based virtual reality scenes', *Journal of Computer Languages*, 71, 101140.

Young, E.G., Zhu, P., Caraza-Harter, T. et al. (2019) 'The true cost of containing: a gVisor case study', in *Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19)*.

Zhang, L. and Oney, S. (2020) 'FlowMatic: an immersive authoring tool for creating interactive scenes in virtual reality', in *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. New York: ACM, pp. 342–353. doi:10.1145/3379337.3415824.

Zhang, Q., Fang, C., Xie, Y. et al. (2023) *A survey on large language models for software engineering*. arXiv:2312.15223.

Zheng, X., Wan, Z., Zhang, Y., Chang, R. and Lo, D. (2024) 'A closer look at the security risks in the Rust ecosystem', *ACM Transactions on Software Engineering and Methodology*, 33(2), Article 34. doi:10.1145/3624738.